Section 1

This program cracks passwords for six user accounts by testing words from a dictionary list against stored password fingerprints. The program first takes the shadow and dictionary files and puts them into lookupable dictionaries. Then it takes the users' hexes and gives it possible hexes based on the length. The program first runs crack_plain, which goes through every word in the dictionary and tries to match it to the fingerprint string. If none of the programs match the plain hexes, the program then tries if any of the hexes have been Caesar-shifted. Cracked_caesar checks by going through 0-25 with all the dictionary words and checks if the fingerprint string matches. Next, it checks if it has been leet-shifted by first generating a leet table. The helper method generate_leet_variations generates all possible combinations of the leet table words. Next, Leet_cracker goes through all the combinations of the leet_variations and checks against all options in the dictionary. Finally, cracked_salted figures out which scrambling method was used, pulls the dictionary words into a plain list, then for each word, scrambles the word by itself first and saves that partial scrambling state using .copy(). For each word, it tries the 00000 to 99999, builds it as a zero-padded 5-digit string, finishes scrambling the word-plus-code combination off the saved starting point, and checks it against the target fingerprint. As soon as a match is found, it returns the original word.

First Attempt

Cracked: user1:wrestlemania (0.08s)

Cracked: user2:snowball1 (178.50s)

Cracked: user3:galadriel (1.18s)

Could not crack: user4 (1265.25s)

Cracked: user5:josephine (0.05s)

Could not crack: user6 (1203.71s)

The first few attempts took extremely long, 40-some minutes for everything to run. Moreover, a few were not cracked, like user4 and user6. And I had no idea why. But one reason was that SHA-256 and SHA3-256 both produce a 64-char hash.

Section 2

To solve this task, I first put the encrypted message into a decoder at this decoder, which told me to use a monoalphabetic substitution which maps the frequency of the encoded message and tries to figure out the message based on frequency. I did this by first taking the frequency of all the letters and remembering where each of the letters is. Then I mapped each of the letter groups to the word that it could be. It is decoded by taking the highest frequency and converting it into the most commonly appearing letters. That generates a frequency table, as seen in CIPHER_TO_PLAIN, and by using the cipher to plain it, it generates a deciphered text.